

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

Automatización de pruebas con Python

Tabla de contenido

1. Introducción	2
2. Entorno de trabajo y herramientas	2
2.1. Entorno utilizado:	2
2.2. Automatización con Makefile y Docker	3
3. Estructura del proyecto	3
4. Implementación de funcionalidades	5
4.1. Clase Calculator (app/calc.py)	5
4.2. Clase API (app/api.py)	7
4.3. Módulo de utilidades (app/util.py)	9
4.4. Pruebas unitarias (test/unit)	10
4.5. Pruebas de API (test/rest)	14
4.5.1. Estructura del fichero api_test.py y diseño de la batería	14
4.5.2. Casos de éxito (HTTP 200): operaciones cubiertas	16
4.5.3. Casos de error (HTTP 400): validación de entradas inválidas	16
4.5.4. Ejecución automática con make test-api y generación de informes	17
4.5.5. Comprobación manual complementaria	18
5. Problemas encontrados	19
6. Conclusiones	20

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

1. Introducción

En esta practica se trabaja con una aplicación sencilla (una calculadora en Python 3) cuyo objetivo no es tanto construir una calculadora “compleja” sino practicar la **diferencia entre pruebas unitarias y pruebas de servicio**, y sobre todo **automatizar** la ejecución de dichas pruebas.

La base del ejercicio parte de un repositorio que incluye una implementación incompleta y una batería de pruebas insuficiente. Se han completado las funcionalidades y ampliado los tests para cubrir **casos de éxito y casos de error**, además de generar informes en formato XML como evidencia de los resultados.

Como se nos ha explicado en la asignatura, la automatización de pruebas reduce el trabajo manual, acelera ciclos y permite repetir pruebas de forma consistente en diferentes entornos.

2. Entorno de trabajo y herramientas

2.1. Entorno utilizado:

- Sistema operativo: **Windows** (entorno local).
- Editor: **Visual Studio Code**.
- Contenerización: **Docker Desktop** para ejecutar la aplicación y las pruebas sin instalar dependencias en el sistema operativo.
- Ejecución de pruebas: **pytest** (junto con **unittest** en algunos tests).
- Framework de API: **Flask**.

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

2.2. Automatización con Makefile y Docker

El proyecto incluye un **Makefile** con distintos objetivos (targets) como `build`, `test-unit` y `test-api` que agrupan comandos y permiten ejecutar tareas repetibles. El objetivo es usar un Makefile como “lista de alias” que dispara contenedores efímeros para pruebas unitarias y pruebas de servicio, generando informes Junit XML.

El flujo seguido ha sido:

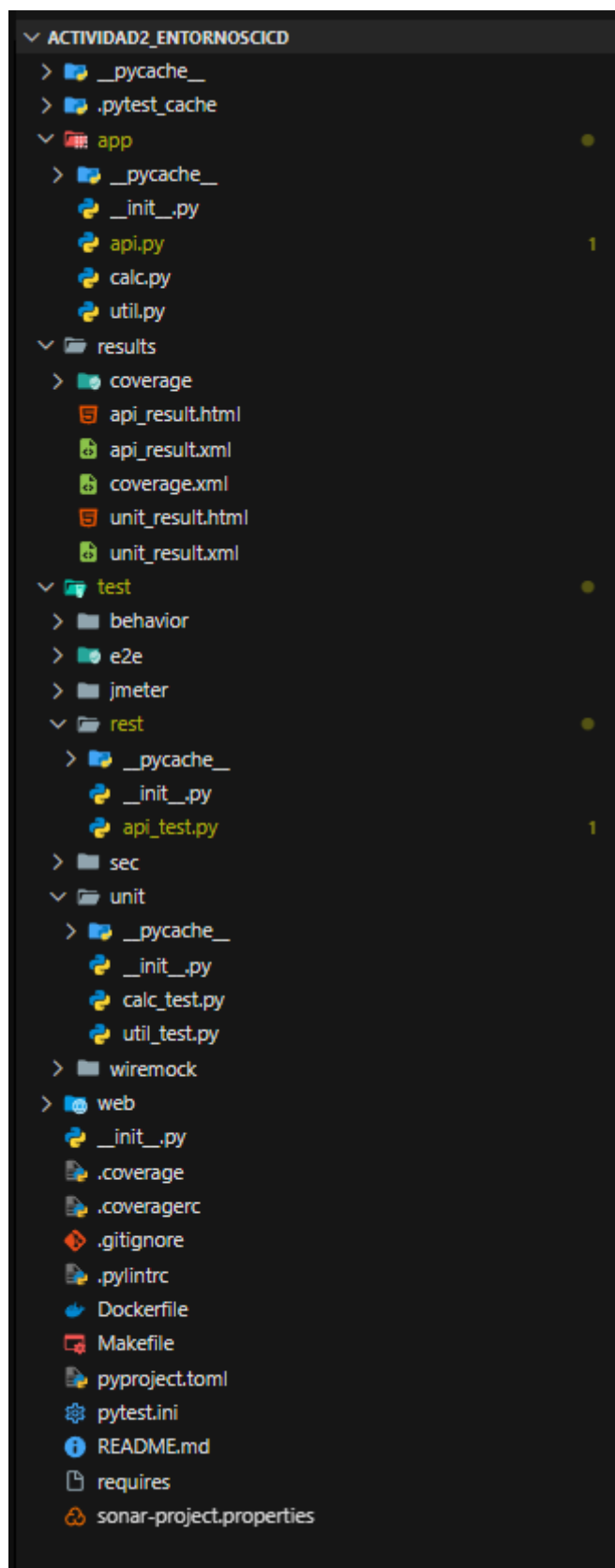
- **make build:** construye la imagen Docker con las dependencias.
- **make test-unit:** ejecuta las pruebas unitaris en un contenedor y genera XML/HTML y cobertura.
- **make test-api:** levanta la API en un contenedor dentro de una red Docker, ejecuta pruebas contra la API con `BASE_URL` y genera XML/HTML.

3. Estructura del proyecto

El repositorio se organiza en varios componentes:

- **app/calculator.py:** clase Calculator con la lógica de operaciones.
- **app/api.py:** API REST (Flask) con endpoints `/calc/<operación>/...` que invocan a Calculator.
- **app/util.py:** funciones auxiliares, especialmente conversión de parámetros a número, etc.
- **test/unit/*:** pruebas unitarias (métodos y utilidades).
- **test/rest/*:** pruebas de servicio/API que realizan peticiones HTTP.
- **results/:** carpeta donde se generan los informes XML/HTML y reportes de cobertura.

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	



Directorio del proyecto

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

4. Implementación de funcionalidades

Esta sección describe los cambios realizados para cumplir el requisito de implementar todas las operaciones solicitadas y controlar los parámetros inválidos.

4.1. Clase Calculator (app/calc.py)

La clase `Calculator` contiene las siguientes funciones:

- **Método `add(x, y)`:**

Este método realiza la suma de dos valores numéricos. Antes de realizar la operación, verifica que ambos parámetros sean números válidos mediante una comprobación de tipos.

```
def add(self, x, y):
    self.check_types(x, y)
    return x + y
```

- **Método `subtract(x, y)`:**

Este método calcula la resta de dos números. Al igual que el caso anterior, incluye una validación de tipos para asegurar que los parámetros son numéricos.

```
def subtract(self, x, y):
    self.check_types(x, y)
    return x - y
```

- **Método `multiply(x, y)`:**

Este método realiza la división entre dos números. Además de la validación de tipos, incluye una comprobación adicional de permisos mediante una función auxiliar.

```
def multiply(self, x, y):
    if not app.util.validate_permissions(f"{x} * {y}", "user1"):
        raise InvalidPermissions("User has no permissions")

    self.check_types(x, y)
    return x * y
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

- **Método `divide(x, y)`:**

Este método realiza la división entre dos números. Incluye una validación específica para evitar divisiones por cero, lanzando una excepción en ese caso.

```
def divide(self, x, y):
    self.check_types(x, y)
    if y == 0:
        raise TypeError("Division by zero is not possible")
    return x / y
```

- **Método `power(x, y)`:**

Este método implementa la operación de potenciación. Devuelve el resultado de elevar el primer operando al segundo.

```
def power(self, x, y):
    self.check_types(x, y)
    return x ** y
```

Métodos añadidos durante la práctica:

- **Método `sqrt(x)`:**

Este método calcula la raíz cuadrada de un número utilizando el módulo `math`. Se ha añadido validación para evitar valores negativos, lanzando una excepción en caso de entrada no válida.

```
def sqrt(self, x):
    self.check_number(x)
    if x < 0:
        raise TypeError("Square root of negative number is not possible")
    return math.sqrt(x)
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

- **Método `log10(x)`:**

Este método calcula el logaritmo en base 10 de un número. Se ha implementado validación para evitar valores menores o iguales a cero.

```
def log10(self, x):
    self.check_number(x)
    if x <= 0:
        raise TypeError("Log10 of zero or negative number is not possible")
    return math.log10(x)
```

- **Método de validación (`check_types` y `check_number`):**

Estos métodos se utilizan para asegurar que los parámetros son numéricos antes de realizar cualquier operación.

```
def check_number(self, x):
    if not isinstance(x, (int, float)):
        raise TypeError("Parameters must be numbers")

def check_types(self, x, y):
    if not isinstance(x, (int, float)) or not isinstance(y, (int, float)):
        raise TypeError("Parameters must be numbers")
```

4.2. Clase API (`app/api.py`)

El módulo `api.py` implementa una API REST utilizando Flask, que permite acceder a las funcionalidades de la clase `Calculator` a través de peticiones HTTP.

- **Endpoint `/calc/add/<op1>/<op2>`**

Este endpoint recibe dos parámetros y devuelve el resultado de su suma.

Incluye gestión de errores para devolver HTTP 400 cuando los parámetros no son válidos.

```
@api_application.route("/calc/add/<op_1>/<op_2>", methods=["GET"])
def add(op_1, op_2):
    try:
        num_1, num_2 = util.convert_to_number(op_1), util.convert_to_number(op_2)
        return ("{}".format(CALCULATOR.add(num_1, num_2)), http.client.OK, HEADERS)
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

- **Endpoint /calc/substract/<op1>/<op2>**

Este endpoint realiza la resta de dos valores y devuelve el resultado en formato texto.

```
@api_application.route("/calc/substract/<op_1>/<op_2>", methods=["GET"])
def substract(op_1, op_2):
    try:
        num_1, num_2 = util.convert_to_number(op_1), util.convert_to_number(op_2)
        return ("{}".format(CALCULATOR.substract(num_1, num_2))), http.client.OK, HEADERS
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

Endpoints añadidos durante la práctica: Para cumplir con el enunciado, se han implementado los siguientes endpoints adicionales.

- **Endpoint /calc/multiply/<op1>/<op2>**

Permite realizar la multiplicación de dos valores a través de la API.

```
@api_application.route("/calc/multiply/<op_1>/<op_2>", methods=["GET"])
def multiply(op_1, op_2):
    try:
        num_1, num_2 = util.convert_to_number(op_1), util.convert_to_number(op_2)
        return ("{}".format(CALCULATOR.multiply(num_1, num_2))), http.client.OK, HEADERS
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

- **Endpoint /calc/divide/<op1>/<op2>**

Realiza la división entre dos valores.

En caso de división por cero, devuelve HTTP 400 con un mensaje de error.

```
@api_application.route("/calc/divide/<op_1>/<op_2>", methods=["GET"])
def divide(op_1, op_2):
    try:
        num_1, num_2 = util.convert_to_number(op_1), util.convert_to_number(op_2)
        return ("{}".format(CALCULATOR.divide(num_1, num_2))), http.client.OK, HEADERS
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

- **Endpoint /calc/power/<op1>/<op2>**

Permite realizar la operación de potenciación a través de la API.

```
@api_application.route("/calc/power/<op_1>/<op_2>", methods=["GET"])
def power(op_1, op_2):
    try:
        num_1, num_2 = util.convert_to_number(op_1), util.convert_to_number(op_2)
        return ("{}".format(CALCULATOR.power(num_1, num_2)), http.client.OK, HEADERS)
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

- **Endpoint /calc/sqrt/<op1>**

Permite calcular la raíz cuadrada de un número.

```
@api_application.route("/calc/sqrt/<op_1>", methods=["GET"])
def sqrt(op_1):
    try:
        num_1 = util.convert_to_number(op_1)
        return ("{}".format(CALCULATOR.sqrt(num_1)), http.client.OK, HEADERS)
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

- **Endpoint /calc/log10/<op1>**

Permite calcular el logaritmo en base 10 de un valor.

```
@api_application.route("/calc/log10/<op_1>", methods=["GET"])
def log10(op_1):
    try:
        num_1 = util.convert_to_number(op_1)
        return ("{}".format(CALCULATOR.log10(num_1)), http.client.OK, HEADERS)
    except TypeError as e:
        return (str(e), http.client.BAD_REQUEST, HEADERS)
```

4.3. Módulo de utilidades (app/util.py)

El módulo util.py contiene funciones auxiliares utilizadas principalmente por la API.

- **Función convert_to_number()**

Esta función convierte los parámetros de entrada (recibidos como texto en la API) a valores numéricos.

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

En caso de que la conversión falle, lanza una excepción que posteriormente es gestionada por la API.

```
def convert_to_number(operand):
    try:
        if "." in operand:
            return float(operand)
        else:
            return int(operand)
    except ValueError:
        raise TypeError("Operator cannot be converted to number")
```

4.4. Pruebas unitarias (test/unit)

Las pruebas unitarias se encargan de validar el comportamiento de los métodos de la clase Calculator y de las funciones auxiliares.

Estas pruebas permiten verificar el correcto funcionamiento del código a nivel interno, sin necesidad de ejecutar la aplicación como servicio.

Aunque el repositorio ya incluía algunas pruebas unitarias, inicialmente la cobertura estaba centrada en operaciones básicas como add, divide y multiply. Se han implementado también potenciación, raíz cuadrada y logaritmo en base 10 cubriendo tanto casos correctos como casos de fallo.

Por este motivo, se amplió el fichero test/unit/calc_test.py añadiendo pruebas específicas para los nuevos métodos de la clase Calculator:

- subtract(x, y): pruebas de resultados correctos y de error cuando los parámetros no son numéricos.
- power(x, y): pruebas para exponentes positivos, cero y negativos, y validación de tipos inválidos.

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

- $\text{sqrt}(x)$: pruebas de éxito (por ejemplo $\text{sqrt}(9) = 3$) y pruebas de error con valores negativos, además de entradas no numéricas. Para evitar falsos fallos por precisión en coma flotante, se utilizó `assertAlmostEqual`.
- $\text{log}_{10}(x)$: pruebas de éxito ($\text{log}_{10}(100) = 2$) y pruebas de error para valores inválidos ($x \leq 0$), además de entradas no numéricas, también usando `assertAlmostEqual` donde corresponde.

Ejecución de `make test-unit`:

```

PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integración y Entrega Continua\Actividades\Actividad2_EntornosCICD> make test-unit
docker run --rm --volume "C:/Users/iandr/Documents/UNIR/Master/2do Cuatrimestre/Entornos Integración y Entrega Continua/Actividades/Actividad2_EntornosCICD:/opt/calc" --env PYTHONP
t pytest --cov --cov-report=xml:results/coverage.xml --cov-report=html:results/coverage --junit-xml=results/unit_result.xml -m unit || true
===== test session starts =====
platform linux -- Python 3.6.15, pytest-5.4.3, py-1.11.0, pluggy-0.13.1
rootdir: /opt/calc, inifile: pytest.ini
plugins: cov-2.10.0
collected 31 items / 12 deselected / 19 selected

test/unit/calc_test.py ..... [ 89%]
test/unit/util_test.py .. [100%]

----- generated xml file: /opt/calc/results/unit_result.xml -----

----- coverage: platform linux, python 3.6.15-final-0 -----
Coverage HTML written to dir results/coverage
Coverage XML written to file results/coverage.xml

===== 19 passed, 12 deselected in 2.08s =====
docker run --rm --volume "C:/Users/iandr/Documents/UNIR/Master/2do Cuatrimestre/Entornos Integración y Entrega Continua/Actividades/Actividad2_EntornosCICD:/opt/calc" --env PYTHONP
t junit2html results/unit_result.xml results/unit_result.html
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integración y Entrega Continua\Actividades\Actividad2_EntornosCICD>

```

En la ejecución final, pytest recogió **31 pruebas**, de las cuales se seleccionaron 19 por la marca unit, y **todas pasaron correctamente** (19 passed, 12 deselected).

Como resultado, se generaron los ficheros:

- `results/unit_result.xml` (informe JUnit XML),
- `results/unit_result.html` (visualización HTML),
- `results/coverage.xml` y carpeta `results/coverage/` (cobertura)

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

Ejemplo de `unit_results.html`:

Test Report : unit_result.xml

```
test.unit.calc_test.TestCalculate
  • test_add_method_fails_with_nan_parameter
  • test_add_method_returns_correct_result
  • test_divide_method_fails_with_division_by_zero
  • test_divide_method_fails_with_nan_parameter
  • test_divide_method_returns_correct_result
  • test_log10_method_fails_with_nan_parameter
  • test_log10_method_fails_with_zero_or_negative_number
  • test_log10_method_returns_correct_result
  • test_multiply_method_fails_with_nan_parameter
  • test_multiply_method_returns_correct_result
  • test_power_method_fails_with_nan_parameter
  • test_power_method_returns_correct_result
  • test_sqrt_method_fails_with_nan_parameter
  • test_sqrt_method_fails_with_negative_number
  • test_sqrt_method_returns_correct_result
  • test_subtract_method_fails_with_nan_parameter
  • test_subtract_method_returns_correct_result
test.unit.util_test.TestUtil
  • test_convert_to_number_correct_param
  • test_convert_to_number_invalid_type
```

Test Suite: pytest

Results

Duration	2.063 sec
Tests	19
Failures	0

Tests

test.unit.calc_test.TestCalculate

Test case:	test_add_method_fails_with_nan_parameter
Outcome:	Passed
Duration:	0.007 sec
Test case:	test_add_method_returns_correct_result

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

Pruebas unitarias añadidas:

```
# MULTIPLY

@patch("app.util.validate_permissions", side_effect=mocked_validation, create=True)
def test_multiply_method_returns_correct_result(self, _validate_permissions):
    self.assertEqual(4, self.calc.multiply(2, 2))
    self.assertEqual(0, self.calc.multiply(1, 0))
    self.assertEqual(0, self.calc.multiply(-1, 0))
    self.assertEqual(-2, self.calc.multiply(-1, 2))

def test_multiply_method_fails_with_nan_parameter(self):
    self.assertRaises(TypeError, self.calc.multiply, "2", 2)
    self.assertRaises(TypeError, self.calc.multiply, 2, "2")
    self.assertRaises(TypeError, self.calc.multiply, "2", "2")

# POWER

def test_power_method_returns_correct_result(self):
    self.assertEqual(8, self.calc.power(2, 3))
    self.assertEqual(1, self.calc.power(2, 0))
    self.assertEqual(0.25, self.calc.power(2, -2))
    self.assertEqual(9, self.calc.power(-3, 2))

def test_power_method_fails_with_nan_parameter(self):
    self.assertRaises(TypeError, self.calc.power, "2", 2)
    self.assertRaises(TypeError, self.calc.power, 2, "2")
    self.assertRaises(TypeError, self.calc.power, None, 2)

# Sqrt

def test_sqrt_method_returns_correct_result(self):
    self.assertAlmostEqual(3.0, self.calc.sqrt(9), places=7)
    self.assertAlmostEqual(0.0, self.calc.sqrt(0), places=7)
    self.assertAlmostEqual(1.41421356237, self.calc.sqrt(2), places=9)

def test_sqrt_method_fails_with_negative_number(self):
    self.assertRaises(TypeError, self.calc.sqrt, -1)
    self.assertRaises(TypeError, self.calc.sqrt, -0.0001)

def test_sqrt_method_fails_with_nan_parameter(self):
    self.assertRaises(TypeError, self.calc.sqrt, "9")
    self.assertRaises(TypeError, self.calc.sqrt, None)
    self.assertRaises(TypeError, self.calc.sqrt, object())

# LOG10

def test_log10_method_returns_correct_result(self):
    self.assertAlmostEqual(2.0, self.calc.log10(100), places=7)
    self.assertAlmostEqual(0.0, self.calc.log10(1), places=7)
    self.assertAlmostEqual(1.0, self.calc.log10(10), places=7)

def test_log10_method_fails_with_zero_or_negative_number(self):
    self.assertRaises(TypeError, self.calc.log10, 0)
    self.assertRaises(TypeError, self.calc.log10, -1)

def test_log10_method_fails_with_nan_parameter(self):
    self.assertRaises(TypeError, self.calc.log10, "100")
    self.assertRaises(TypeError, self.calc.log10, None)
    self.assertRaises(TypeError, self.calc.log10, object())
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

4.5. Pruebas de API (test/rest)

Las pruebas de API validan el comportamiento de la aplicación desde el punto de vista de un cliente externo, realizando peticiones HTTP reales a los distintos endpoints.

En este proyecto las pruebas de API se encuentran en el fichero `test/rest/api_test.py`. Su ejecución se realiza con `pytest` usando la marca `api` y, por tanto, se pueden separar en una batería independiente (en mi caso ejecutadas mediante `make test-api`). Esta separación por baterías es útil para pipelines CI/CD porque permite ejecutar unitarias y servicio en fases distintas y generar informes homogéneos en formato JUnit XML.

4.5.1. Estructura del fichero `api_test.py` y diseño de la batería

El fichero `api_test.py` está implementado con `unittest.TestCase` y decorado con `@pytest.mark.api`, de manera que `pytest -m api` seleccione únicamente esta batería. En el método `setUp()` se comprueba que la variable de entorno `BASE_URL` está correctamente configurada, ya que las pruebas dependen de una URL base para realizar las peticiones HTTP a la API.

Para evitar repetición de código, se incluyó un helper privado `_get(path)` que:

1. Construye la URL final concatenando `BASE_URL` y la ruta.
2. Lanza la petición con `urlopen`.
3. Devuelve un par `(status_code, body_text)`.

Este helper también gestiona explícitamente el caso en el que la API devuelve códigos no-2xx (por ejemplo 400), ya que `urlopen` lanza `HTTPError`. En estos casos se captura la excepción y se devuelve igualmente el código y el cuerpo de respuesta. Esto permite escribir tests que comprueben **tanto el status code (400)** como el mensaje

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

textual devuelto por la API, algo especialmente importante cuando se validan condiciones de error.

```
@pytest.mark.api
class TestApi(unittest.TestCase):
    def setUp(self):
        self.assertIsNotNone(BASE_URL, "BASE_URL no configurada")
        self.assertTrue(len(BASE_URL) > 8, "BASE_URL no configurada")

    def _get(self, path):
        """
        Devuelve (status_code, body_text).
        Si la API responde 400/404..., urlopen lanza HTTPError: lo capturamos.
        """
        url = f"{BASE_URL}{path}"
        try:
            resp = urlopen(url, timeout=DEFAULT_TIMEOUT)
            body = resp.read().decode("utf-8")
            return resp.status, body
        except HTTPError as e:
            body = e.read().decode("utf-8")
            return e.code, body
        except URLError as e:
            self.fail(f"No se pudo conectar a la API en {url}. Error: {e}")

# ----- CASOS OK (200) -----
def test_api_add_ok(self):
    status, body = self._get("/calc/add/2/2")
    self.assertEqual(status, http.client.OK)
    self.assertEqual(body, "4")

def test_api_substract_ok(self):
    status, body = self._get("/calc/substract/5/3")
    self.assertEqual(status, http.client.OK)
    self.assertEqual(body, "2")

def test_api_multiply_ok(self):
    status, body = self._get("/calc/multiply/2/4")
    self.assertEqual(status, http.client.OK)
    self.assertEqual(body, "8")

def test_api_divide_ok(self):
    status, body = self._get("/calc/divide/8/2")
    self.assertEqual(status, http.client.OK)
    self.assertIn(body, ["4", "4.0"])

def test_api_power_ok(self):
    status, body = self._get("/calc/power/2/3")
    self.assertEqual(status, http.client.OK)
    self.assertIn(body, ["8", "8.0"])

def test_api_sqrt_ok(self):
    status, body = self._get("/calc/sqrt/9")
    self.assertEqual(status, http.client.OK)
    self.assertIn(body, ["3", "3.0"])

def test_api_log10_ok(self):
    status, body = self._get("/calc/log10/100")
    self.assertEqual(status, http.client.OK)
    self.assertIn(body, ["2", "2.0"])
```

```
# ----- CASOS KO (400) -----
def test_api_divide_by_zero_returns_400(self):
    status, body = self._get("/calc/divide/1/0")
    self.assertEqual(status, http.client.BAD_REQUEST)
    self.assertIn("Division by zero", body)

def test_api_sqrt_negative_returns_400(self):
    status, body = self._get("/calc/sqrt/-1")
    self.assertEqual(status, http.client.BAD_REQUEST)
    self.assertIn("Square root of negative", body)

def test_api_log10_zero_returns_400(self):
    status, body = self._get("/calc/log10/0")
    self.assertEqual(status, http.client.BAD_REQUEST)
    self.assertIn("Log10 of zero", body)

def test_api_non_numeric_returns_400(self):
    status, body = self._get("/calc/add/a/2")
    self.assertEqual(status, http.client.BAD_REQUEST)
    self.assertIn("cannot be converted", body)
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

4.5.2. Casos de éxito (HTTP 200): operaciones cubiertas

Se añadieron pruebas para cubrir los endpoints principales con casos correctos (HTTP 200). En concreto, los tests implementados son:

- **test_api_add_ok()** → valida `/calc/add/2/2` y espera resultado "4".
- **test_api_subtract_ok()** → valida `/calc/substract/5/3` y espera resultado "2".
- **test_api_multiply_ok()** → valida `/calc/multiply/2/4` y espera resultado "8".
- **test_api_divide_ok()** → valida `/calc/divide/8/2` y verifica HTTP 200; el cuerpo se comprueba de forma tolerante ("4" o "4.0") por el formateo de floats.
- **test_api_power_ok()** → valida `/calc/power/2/3` (resultado "8" o "8.0").
- **test_api_sqrt_ok()** → valida `/calc/sqrt/9` (resultado "3" o "3.0").
- **test_api_log10_ok()** → valida `/calc/log10/100` (resultado "2" o "2.0").

Esta cobertura asegura que la API implementa y expone correctamente todas las operaciones requeridas por el enunciado (suma, resta, multiplicación, división, potenciación, raíz cuadrada y log10).

4.5.3. Casos de error (HTTP 400): validación de entradas inválidas

Además de los casos correctos, se han implementado pruebas de error (devuelven **HTTP 400 Bad Request**) que verifican:

- **test_api_divide_by_zero_returns_400()** → Comprueba que `/calc/divide/1/0` devuelve 400 y que el cuerpo contiene "Division by zero".
- **test_api_sqrt_negative_returns_400()** → Comprueba que `/calc/sqrt/-1` devuelve 400 y que el cuerpo contiene el mensaje asociado a raíz de negativo.

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

- `test_api_log10_zero_returns_400()` → Comprueba que `/calc/log10/0` devuelve 400 y que el cuerpo contiene el mensaje asociado a log10 inválido.
- `test_api_non_numeric_returns_400()` → Comprueba que `/calc/add/a/2` devuelve 400 y que el cuerpo contiene un mensaje indicando que no se puede convertir a número.

Estos tests son importantes porque validan el comportamiento “externo” del sistema: no basta con que internamente la calculadora lance `TypeError`, sino que la API debe traducir ese error a un código HTTP adecuado (400), manteniendo coherencia y facilitando el consumo por clientes.

4.5.4. Ejecución automática con `make test-api` y generación de informes (JUnit XML)

La batería de API se ejecutó de forma automatizada mediante: **`make test-api`**

Este target crea una red Docker efímera, levanta un contenedor `apiserver` con Flask y ejecuta `pytest -m api` en un contenedor separado apuntando a `BASE_URL=http://apiserver:5000/`.

Como evidencia de ejecución, `pytest` genera un informe en formato **JUnit XML**: `results/api_result.xml` y posteriormente se convierte a **HTML**: `results/api_result.html`

En la ejecución final se seleccionaron 11 pruebas marcadas como `api`, completando todas correctamente (11 passed).

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

```
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD> make test-api
docker network create calc-test-api || exit 0
f0c29886f6d943e584fe9a312c2323f207199d9d6ee0fd64f6238ff8a88d1e
docker run -d --rm --volume "C:/Users/iandr/Documents/UNIR/Master/2do Cuatrimestre/Entornos Integracion y Entrega Continua/Actividades/Actividad2_EntornosCICD:/opt/calc" --network calc-test-api --env PYTHONPATH=/opt/calc --env FLASK_APP=app/api.py -p 5000:5000 --w /opt/calc calculator-app:latest flask run --host=0.0.0.0
13416a114aa3e5a07cfd5680cf48d2d45a0b71c32207720e9c33a0d39a16
docker run --rm --volume "C:/Users/iandr/Documents/UNIR/Master/2do Cuatrimestre/Entornos Integracion y Entrega Continua/Actividades/Actividad2_EntornosCICD:/opt/calc" --network calc-test-api --env PYTHONPATH=/opt/calc --env FLASK_APP=app/api.py -p 5000:5000 --w /opt/calc calculator-app:latest pytest --junit-xml=results/api_result.xml -m api || exit 0
===== test session starts =====
platform linux -- Python 3.6.15, pytest-5.4.3, py-1.11.0, pluggy-0.13.1
rootdir: /opt/calc, inifile: pytest.ini
plugins: pytest-7.0.0
collected 31 items / 20 deselected / 11 selected

test/rest/api_test.py ..... [100%]

----- generated xml file: /opt/calc/results/api_result.xml -----
===== 11 passed, 20 deselected in 1.635s =====
docker run --rm --volume "C:/Users/iandr/Documents/UNIR/Master/2do Cuatrimestre/Entornos Integracion y Entrega Continua/Actividades/Actividad2_EntornosCICD:/opt/calc" --env PYTHONPATH=/opt/calc --w /opt/calc calculator-app:latest pytest --junit-xml=results/api_result.xml
docker stop apiserver || exit 0
apiserver
docker network rm calc-test-api || exit 0
calc-test-api
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD>
```

4.5.5. Comprobación manual complementaria

Como comprobación adicional (sin sustituir a la automatización), se han validado manualmente algunas pruebas lanzando `make server` en un terminal y peticiones en otro con el siguiente formato:

`curl.exe -i http://localhost:5000/calc/divide/1/0`

```
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD> make server
docker run --rm --volume "C:/Users/iandr/Documents/UNIR/Master/2do Cuatrimestre/Entornos Integracion y Entrega Continua/Actividades/Actividad2_EntornosCICD:/opt/calc" --env PYTHONPATH=/opt/calc --w /opt/calc calculator-app:latest flask run --host=0.0.0.0
* Serving Flask app "app/api.py"
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: off
* Running on all addresses.
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://172.17.0.2:5000/ (Press CTRL+C to quit)
```

```
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD> curl.exe -i "http://localhost:5000/calc/sqrt/2"
Content-Type: text/plain
Access-Control-Allow-Origin: *
Content-Length: 18
Server: Werkzeug/2.0.3 Python/3.6.15
Date: Mon, 18 May 2026 15:07:21 GMT

1.4142135623730951
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD> curl.exe -i "http://localhost:5000/calc/log10/100"
HTTP/1.0 200 OK
Content-Type: text/plain
Access-Control-Allow-Origin: *
Content-Length: 3
Server: Werkzeug/2.0.3 Python/3.6.15
Date: Mon, 18 May 2026 15:07:32 GMT

2.0
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD> curl.exe -i "http://localhost:5000/calc/log10/0"
HTTP/1.0 400 BAD REQUEST
Content-Type: text/plain
Access-Control-Allow-Origin: *
Content-Length: 48
Server: Werkzeug/2.0.3 Python/3.6.15
Date: Mon, 18 May 2026 15:07:44 GMT

Log10 of zero or negative number is not possible
PS C:\Users\iandr\Documents\UNIR\Master\2do Cuatrimestre\Entornos Integracion y Entrega Continua\Actividades\Actividad2_EntornosCICD> curl.exe -i "http://localhost:5000/calc/sqrt/-1"
HTTP/1.0 400 BAD REQUEST
Content-Type: text/plain
Access-Control-Allow-Origin: *
Content-Length: 46
Server: Werkzeug/2.0.3 Python/3.6.15
Date: Mon, 18 May 2026 15:07:55 GMT

Square root of negative number is not possible
```

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

Las respuestas de errores muestran HTTP/1.0 400 BAD REQUEST y un mensaje explicativo en el cuerpo, lo que confirma que el comportamiento observado manualmente coincide con lo cubierto por `test_api_divide_by_zero_returns_400()`.

5. Problemas encontrados

Durante el desarrollo de la práctica se han encontrado varios problemas relacionados principalmente con el entorno de ejecución y la adaptación del proyecto a Windows.

En primer lugar, durante la construcción de la imagen Docker (`make build`) se produjeron errores de conexión al intentar descargar la imagen base `python:3.6-slim`. Este problema estaba relacionado con condiciones de red externas (problemas con cloflare en horario de retransmisión de La Liga), por lo que la solución fue reintentar la operación en otro momento y continuar trabajando en paralelo con el código y los tests hasta recuperar la conectividad.

Otro de los problemas principales fue la incompatibilidad del Makefile con Windows, ya que estaba orientado a sistemas tipo Linux. Esto generó errores con el uso de `pwd`, el parámetro `--volume`, y comandos como `|| true`, que no existen en PowerShell. Para solucionarlo, se adaptaron los comandos utilizando rutas absolutas compatibles con Docker (`C:/...`) y corrigiendo la sintaxis de los volúmenes (`--volume`). Con estas modificaciones se consiguió ejecutar correctamente tanto `make test-unit` como `make test-api`.

También se detectó un comportamiento inesperado al usar `curl` en PowerShell, ya que en realidad es un alias de `Invoke-WebRequest`. Esto hacía que las respuestas HTTP 400 se interpretaran como errores en lugar de resultados válidos. La solución fue usar `curl.exe`, lo que permitió comprobar correctamente los códigos HTTP y las respuestas de la API.

Asignatura	Datos del alumno	Fecha
Entornos Integración y Entrega Continua	Apellidos: Andreu Orta	18/05/2026
	Nombre: Ignacio	

Por último, se produjeron errores relacionados con Docker, como el uso incorrecto de `--network-alias` sin una red definida o la eliminación manual de contenedores que ya se estaban borrando automáticamente con `--rm`. Estos problemas se resolvieron ajustando los comandos del Makefile para simplificar la ejecución y evitar errores innecesarios.

6. Conclusiones

Esta práctica ha sido especialmente útil desde un punto de vista práctico, ya que hasta el momento no había trabajado con pruebas de software en un contexto real (ni cuento experiencia previa en entornos de integración continua o automatización de pipelines).

Uno de los aspectos más interesantes ha sido entender de forma clara la diferencia entre pruebas unitarias y pruebas de API.

También ha resultado relevante la implementación del control de errores, especialmente en la API, donde no basta con lanzar excepciones, sino que es necesario traducirlas a respuestas HTTP correctas.

A nivel de entorno, se ha decidido trabajar en Windows en lugar de utilizar una máquina virtual Linux, precisamente para enfrentarme a problemas reales de compatibilidad que puedan surgir. Esto ha obligado a adaptar el Makefile y los comandos de Docker, lo que ha permitido entender mejor cómo funcionan estos mecanismos y qué diferencias existen entre entornos.

En general, la práctica ha permitido consolidar conocimientos sobre pruebas de software, entender la importancia de automatizar la ejecución y obtener evidencias en formato estándar (JUnit XML), y aplicar estos conceptos en un escenario que se aproxima bastante a lo que se podría encontrar en un entorno profesional.